Kathmandu University

Department of Computer Science and Engineering
Dhulikhel, Kavre

A Mini Project Report

on

"ZOMBIE HUNT (FIFO SURVIVAL GAME)"

[Code No.: COMP 202]

(For partial fulfillment of Year II / Semester I in Computer Engineering)

Submitted by

Rhijvi Dhungel
Roll number: 68

Submitted to

Mr. Sagar Acharya
Department of Computer Science and Engineering

February 25, 2026

Acknowledgement

I would like to express my sincere gratitude to the **"Department of Computer Science and Engineering"** for providing me with this remarkable platform and the necessary resources to carry out this mini project. This opportunity has broadened my perspective and enabled me to apply theoretical knowledge in a practical manner, connecting it to real-world applications.

My utmost thanks goes to our lecturer, **Mr. Sagar Acharya** for his valuable guidance, continuous encouragement, and clear direction throughout the semester. I also extend our heartfelt thanks to all faculty members and staff for creating a positive and supportive environment that encouraged improved my productivity.

Furthermore, I gratefully acknowledge the constant support and motivation provided by our family members and friends, who played a vital role in the successful completion of this mini project.

This mini project has provided me with hands-on experience in problem-solving, and proper understanding of topic, which has contributed significantly to mygrowth in all aspect.

Rhijvi Dhungel
Roll number: 68

Abstract

A zombie survival game is designed to simulate a structured enemy elimination system based on queue (FIFO) principles. The system manages multiple waves of zombies where players must eliminate enemies in a specific order to survive. This project focuses on the development of a desktop-based 2D game that integrates data structures, real-time interaction, and database management.

The application is developed using **C programming language with the Raylib graphics library** and incorporates **SQLite database integration** for user authentication and high-score management. The proposed system follows a structured gameplay mechanism in which zombies are managed using **queue-based logic (FIFO)**, where only the front zombie in the queue can be eliminated at a given time. As levels progress, gameplay difficulty increases through bullet limitations, obstacle movement, boss enemies, and baby zombie spawning mechanisms.

The **user authentication module** enables player registration and login using secure password hashing techniques. The **gameplay module** controls zombie spawning, queue management, collision detection, level progression, and sound effects. The **database module** maintains user records and high scores, ensuring persistent data storage.

This project demonstrates the practical implementation of data structures (Queue), linked lists (for bullet management), file/database handling, graphics rendering, and real-time event-driven programming in game development.

Keywords: Zombie Game, Queue (FIFO), Raylib, C Programming, SQLite, Game Development, Data Structures, Survival Game

Contents

Acknowledgement **i**

Abstract **ii**

Table of Contents **iv**

List of Figures **v**

List of Tables **1**

1 Introduction **2**

1.1 Background 2

1.2 Objectives 3

1.3 Motivation and Significance 3

2 Literature Review **4**

2.1 Introduction 4

2.2 Previous Works 4

2.3 Comparison of Existing Systems 5

2.4 Current Work 5

2.5 Overview of Existing Systems 5

2.6 Technologies Used in Previous Works 6

2.7 Drawbacks and Gaps in Existing Systems 6

2.8 Summary 6

3 Methodology **7**

3.1 System Overview 7

3.2 System Architecture 8

3.3 Major Modules 9

3.3.1 Login and Authentication Module 9

3.3.2 Gameplay Module 9

3.3.3 Queue-Based Zombie Management 10

3.3.4 Bullet Management (Linked List) 11
3.3.5 Collision Detection Module 11
3.3.6 Level and Difficulty System 12
3.4 Game States 12
3.4.1 Game Over Screen 12
3.4.2 Victory Screen 14
3.5 Database Design 14
3.6 Tools and Technologies Used 15
3.7 Summary 15

4 Discussion and Achievements 16

5 System Requirements 20
5.1 Software Requirements 20
5.1.1 Minimum Software Requirements 20
5.2 Hardware Requirements 20
5.2.1 Minimum Hardware Requirements 20
5.3 Functional Requirements 21
5.4 Non-Functional Requirements 22
5.4.1 Performance Requirements 22
5.4.2 Security Requirements 22
5.4.3 Usability and Accessibility 22
5.4.4 Maintainability and Scalability 22

6 Conclusion and Recommendation 23
6.1 Limitations 23
6.2 Future Enhancements 24

References 25

Appendix 26
.1 Additional Figures and Screenshots 26

List of Figures

3.1 System Flowchart of Zombie Hunt (FIFO Survival) 8
3.2 Login Screen of Zombie Hunt 9
3.3 Main Gameplay Screen 10
3.4 Baby Zombies Spawned After Boss Defeat 11
3.5 Game Over Screens 13
3.6 Victory Screen – Congratulations Message 14

A1 Login Screen of Zombie Hunt (FIFO Survival Game) 26
A2 Gameplay - Level 1 (Unlimited Bullets) 27
A3 Gameplay - Level 2 (Limited Bullets) 27
A4 Gameplay - Level 3 (Increased Difficulty) 28
A5 Gameplay - Level 4 (Moving Obstacles) 28
A6 Gameplay - Level 5 (High Intensity Mode) 29
A7 Gameplay - Level 6 (Final Survival Stage) 29
A8 Baby Zombies Spawned After Boss Defeat 30
A9 Game Over Screen when zombie touches player 30
A10 Game Over Screen when player touches obstacle 31
A11 Congratulations Screen After Completing Level 6 31
A12 System Flowchart of Queue-Based Zombie Survival Game 32

List of Tables

2.1 Comparison of Previous Works 5

Chapter 1

Introduction

1.1 Background

Video games have evolved from simple entertainment tools into powerful platforms for demonstrating programming concepts, data structures, and real-time system design (Cormen, Leiserson, Rivest, & Stein, 2009). Among various game genres, survival and action-based games are particularly effective in illustrating algorithmic logic, event-driven programming, and interactive graphics rendering (Stroustrup, 2013).

Traditional 2D survival games typically allow players to eliminate enemies freely without structured targeting rules. However, such implementations often do not demonstrate deeper data structure applications in gameplay logic. Integrating structured algorithms such as **Queue (FIFO)** management into enemy handling introduces an educational dimension to game development (Cormen et al., 2009). By enforcing a First-In-First-Out elimination rule, the game simulates controlled processing similar to real-world queue systems used in operating systems, network packet handling, and service management.

Modern game development frameworks such as **Raylib** provide lightweight and efficient tools for building real-time graphical applications in C programming (Technologies, 2024). Additionally, database systems like **SQLite** enable persistent data storage for user authentication and score tracking (SQLite Consortium, 2024). Combining real-time rendering, collision detection, linked list management, and database integration creates a practical environment for applying theoretical computer science concepts in an interactive format (Hlavacs, 2017).

Despite the popularity of survival games, many projects focus primarily on visual effects rather than structured algorithm implementation (Nguyen, 2023). This project introduces a **queue-based zombie survival system** where enemies must be eliminated in FIFO order, ensuring that gameplay directly reflects data structure behavior. This approach enhances both educational value and gameplay complexity.

1.2 Objectives

The primary objectives of this project include:

- Implement a **queue-based (FIFO) zombie management system** where only the front zombie can be eliminated.
- Develop a real-time 2D survival game using **C programming and the Raylib graphics library**.
- Integrate a **linked list structure** for dynamic bullet management.
- Implement **user authentication and high-score tracking** using SQLite database integration.
- Design progressive difficulty levels including boss enemies, limited ammunition, and moving obstacles.
- Create an interactive and visually engaging gameplay environment with sound effects and animations.

1.3 Motivation and Significance

The motivation behind developing this project was to demonstrate how theoretical data structures can be applied in a practical and interactive manner. Rather than implementing a traditional survival game, this project emphasizes the educational integration of **Queue (FIFO) logic**, linked lists, and real-time event handling .

This project provides:

- A practical demonstration of **data structures in game development**.
- A structured enemy elimination mechanism based on queue processing principles.
- A secure login system with password hashing using SHA-based cryptographic techniques.
- Persistent storage of high scores using a relational database.
- Progressive difficulty levels culminating in a final survival stage.

Unlike conventional survival games, this system enforces algorithmic discipline through FIFO-based zombie targeting, making it both an entertainment application and a learning tool.

In summary, the implemented system successfully demonstrates the integration of **Queue data structures, linked lists, real-time collision detection, graphics rendering, and database management** within a unified game environment. The project highlights the practical application of core computer science concepts in interactive software development.

Chapter 2

Literature Review

2.1 Introduction

Several 2D survival and zombie-based games have been developed to demonstrate real-time gameplay mechanics, enemy spawning systems, and collision detection techniques. Reviewing these systems helps identify their strengths, weaknesses, and implementation approaches, thereby providing a foundation for the proposed queue-based zombie survival game.

2.2 Previous Works

- H. Hlavacs (2017) proposed a procedurally generated world for a zombie survival game, focusing on dynamic level creation and survival complexity. The study emphasized environmental randomness and increasing difficulty but did not integrate structured data structures such as FIFO queue-based targeting (Hlavacs, 2017).

- T. Nguyen (2023) developed a 3D zombie survival run game using the Unity engine. The project implemented wave-based enemy spawning, boss encounters, and progressive difficulty. While the system demonstrated complete survival mechanics, it did not incorporate algorithm-driven queue processing within gameplay logic (Nguyen, 2023).

- Motion Twin (2008) introduced the multiplayer survival game *Die2Nite*, where players defend against daily zombie attacks and manage survival resources. The game emphasizes strategic survival but does not apply formal data structure concepts such as FIFO elimination logic (Motion Twin, 2008).

Source	Platform	Features	Limitations
Hlavacs (2017) (Hlavacs, 2017)	Procedural Game Research	Dynamic world generation; survival difficulty scaling	No structured queue-based enemy elimination
Nguyen (2023) (Nguyen, 2023)	Unity (3D Game)	Enemy waves; boss fights; level progression	No formal data structure integration
Die2Nite (2008) (Motion Twin, 2008)	Browser Multiplayer Game	Strategic survival; daily zombie waves	No algorithm-driven targeting mechanism

Table 2.1: Comparison of Previous Works

2.3 Comparison of Existing Systems

2.4 Current Work

The proposed system is a **2D Zombie Survival Game developed in C using Raylib and SQLite** that:

- Implements a strict **FIFO (First-In-First-Out) zombie elimination mechanism**.
- Uses a **linked list** structure for dynamic bullet management.
- Integrates **SQLite database** for user login authentication and high-score persistence.
- Applies **SHA-256 password hashing** for secure credential storage.
- Introduces progressive levels, boss zombies, obstacle movement, and bullet limitations.

2.5 Overview of Existing Systems

Zombie survival games have evolved significantly from simple wave-based shooters to complex strategy and multiplayer survival platforms. Most existing systems allow players to eliminate enemies freely without restriction, focusing primarily on reflex-based action mechanics.

Modern frameworks such as Raylib enable efficient 2D graphics rendering, real-time event handling, and sound integration. Game engines such as Unity provide advanced 3D rendering and physics systems. However, most implementations emphasize visual and gameplay elements rather than structured algorithmic processing.

Unlike traditional survival games, the proposed system integrates academic data structure concepts directly into gameplay logic, thereby combining entertainment with algorithm-based control.

2.6 Technologies Used in Previous Works

Most survival games use graphics libraries or engines such as Unity, Unreal Engine, SDL, or Raylib for rendering and game loop management. Enemy spawning and bullet management are typically handled using arrays or dynamic memory allocation.

Persistent data storage in games is commonly implemented using local files or lightweight relational databases such as SQLite. However, secure authentication mechanisms such as SHA-based hashing are often not included in small-scale projects.

Data structures such as Queue (FIFO) and Linked Lists are usually demonstrated in academic console applications rather than integrated into graphical real-time systems.

2.7 Drawbacks and Gaps in Existing Systems

Despite advancements in survival game development, several limitations remain:

- Most zombie survival games do not enforce structured FIFO enemy elimination.
- Data structure concepts are rarely embedded into interactive gameplay mechanics.
- Many small-scale game projects lack secure user authentication systems.
- Persistent high-score systems are often implemented using insecure text files.
- Limited focus on combining real-time graphics with algorithmic discipline.

These limitations highlight the need for a **queue-driven zombie survival system** that integrates structured data processing, secure authentication, database persistence, and real-time graphics rendering.

2.8 Summary

The review of existing literature and survival game systems demonstrates that while numerous zombie survival implementations exist, most focus primarily on gameplay and visual interaction without integrating formal data structure concepts.

The implemented system addresses this gap by combining **FIFO-based zombie targeting, linked list bullet management, secure database integration, and progressive survival mechanics**. Thus, the project serves both as an interactive entertainment application and as a practical demonstration of algorithm implementation within a real-time graphical environment.

Chapter 3

Methodology

3.1 System Overview

The proposed system is a desktop-based 2D survival game titled **"Zombie Hunt: FIFO Survival"**, developed using the C programming language and the Raylib graphics library. The primary objective of the game is to survive waves of zombies while following a strict **Queue (First-In-First-Out)** elimination rule.

Unlike traditional survival games where any enemy can be eliminated freely, this system enforces that only the zombie at the front of the queue can be destroyed. This makes the gameplay directly reflect the behavior of the Queue data structure.

The system integrates:

- Real-time rendering using Raylib
- FIFO-based zombie management
- Linked list for bullet handling
- SQLite database for login and high-score storage
- Multi-level progression with boss enemies

3.2 System Architecture

The game follows a modular architecture. Each module performs an independent task but interacts with others through structured logic and shared data.

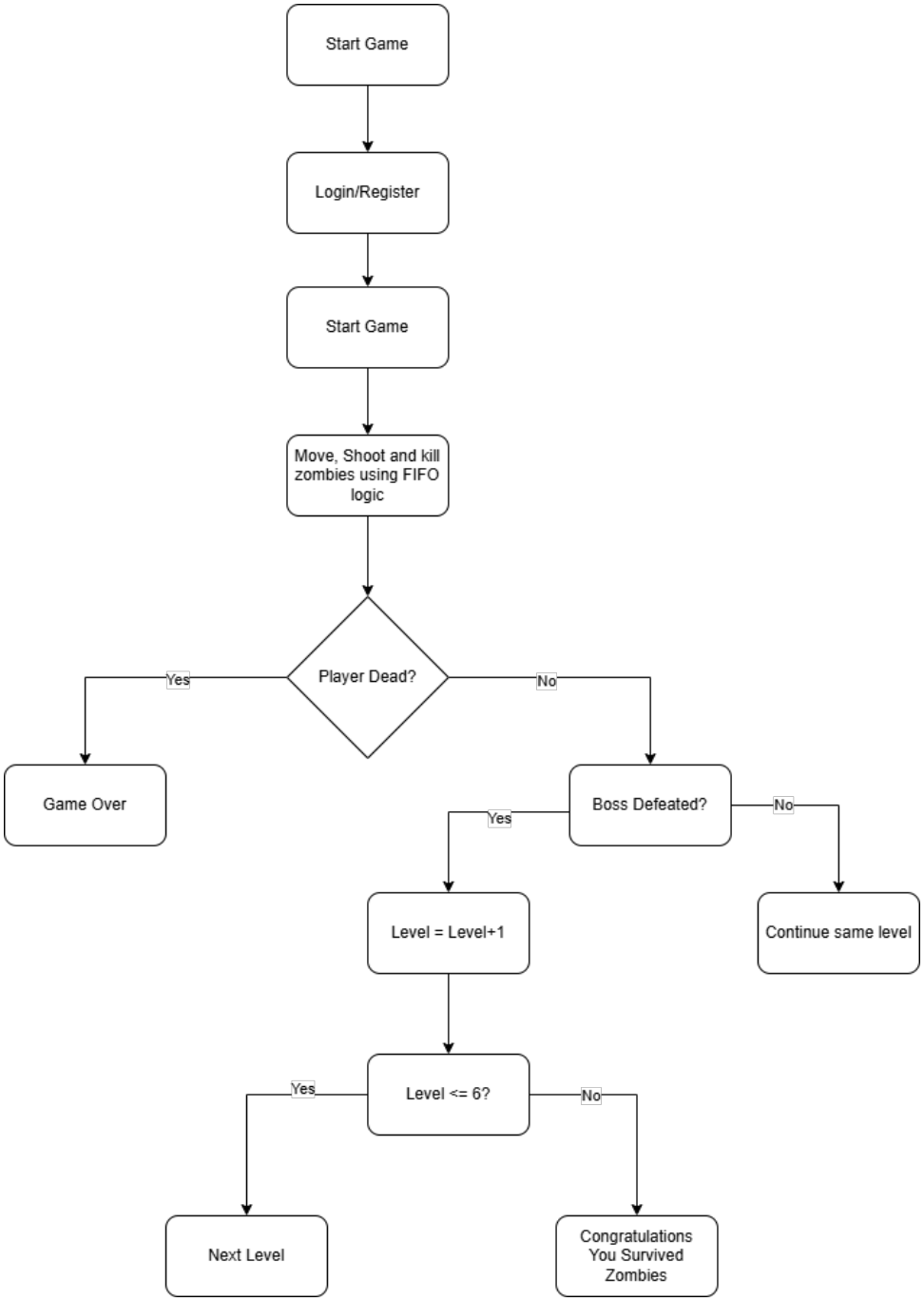

Figure 3.1: System Flowchart of Zombie Hunt (FIFO Survival)

3.3 Major Modules

3.3.1 Login and Authentication Module

This module allows users to register and log in securely. Passwords are hashed using SHA-256 before being stored in the SQLite database. The module retrieves high-score data associated with each user.

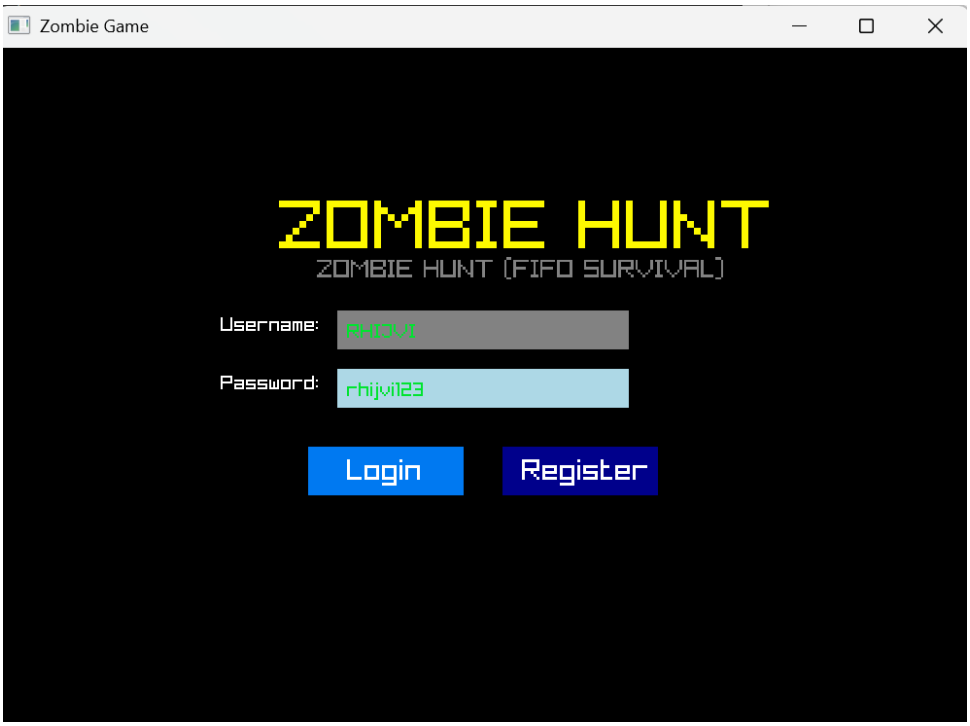

Figure 3.2: Login Screen of Zombie Hunt

3.3.2 Gameplay Module

This module controls the main game loop including:

- Player movement using keyboard input
- Shooting mechanics
- Zombie spawning logic
- Collision detection
- Score updates

The game operates in real-time and updates every frame using Raylib's event-driven architecture.

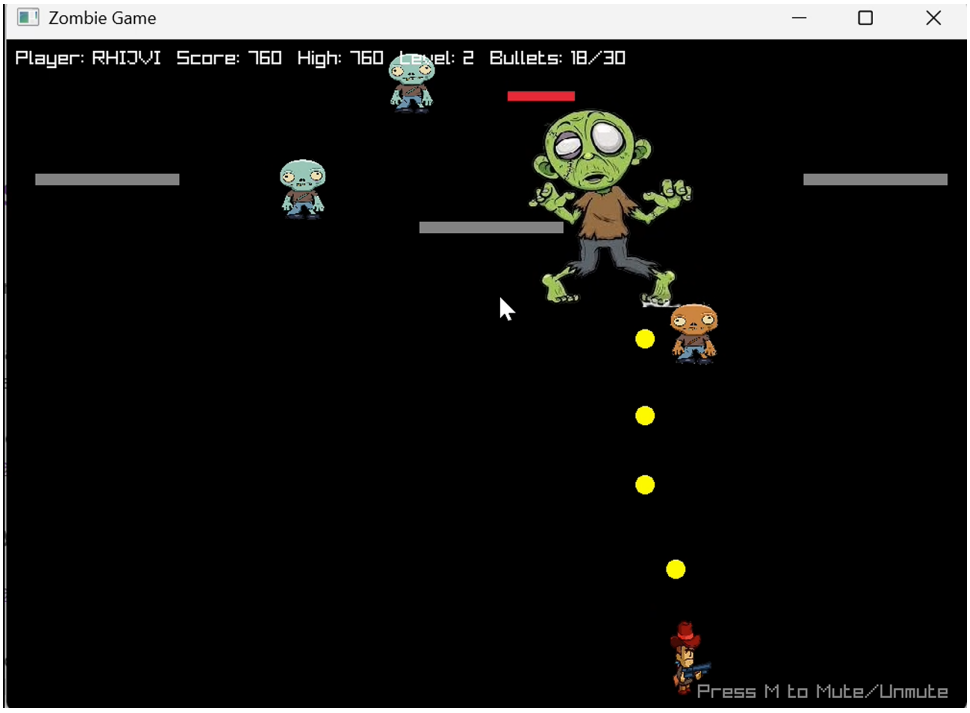

Figure 3.3: Main Gameplay Screen

3.3.3 Queue-Based Zombie Management

Zombies are inserted into a queue when they enter the screen. Only the front zombie (FIFO rule) is allowed to take damage and be eliminated.

Two separate queues are maintained:

- Normal Zombie Queue
- Baby Zombie Queue (spawned after boss defeat)

This structured approach ensures strict data structure implementation within gameplay.

Figure 3.4: Baby Zombies Spawned After Boss Defeat

3.3.4 Bullet Management (Linked List)

Bullets are dynamically created when the player shoots. A singly linked list is used to:

- Store active bullets
- Update bullet movement
- Remove bullets when collision occurs

This allows efficient memory management and dynamic object handling.

3.3.5 Collision Detection Module

The system uses Raylib collision functions:

- `CheckCollisionRecs()` for player-zombie and player-obstacle collision
- `CheckCollisionCircles()` for bullet-zombie collision

If the player collides with a zombie or obstacle, the game transitions to the Game Over state.

3.3.6 Level and Difficulty System

The game contains six progressive levels:

- Level 1: Unlimited bullets
- Level 2-5: Limited bullets with increasing difficulty
- Level 6: Extreme survival stage with only 2 bullets

Every 20 zombie eliminations spawn a Boss Zombie. Defeating the boss increases the level.

3.4 Game States

The game operates under different states:

- Login State
- Playing State
- Game Over State
- Victory State

3.4.1 Game Over Screen

If the player collides with a zombie or obstacle before completing Level 6, the Game Over screen is displayed.

Figure 3.5: Game Over Screens

3.4.2 Victory Screen

After completing Level 6 successfully, the game displays a congratulatory message and stops further enemy rendering.

Figure 3.6: Victory Screen – Congratulations Message

3.5 Database Design

SQLite is used to store:

- User credentials
- Hashed passwords
- High scores

The database ensures persistence across sessions.

3.6 Tools and Technologies Used

- Programming Language: C
- Graphics Library: Raylib
- Database: SQLite
- Cryptographic Library: OpenSSL (SHA-256)
- Compiler: GCC (MinGW)
- Operating System: Windows 10/11

3.7 Summary

This methodology chapter described the architectural design, module structure, queue-based zombie management system, collision detection, database integration, and level progression logic of the proposed game.

The system demonstrates the practical implementation of data structures such as Queue and Linked List within a real-time interactive game environment, combining both educational value and entertainment.

Chapter 4

Discussion and Achievements

Project Overview

The Queue-Based Zombie Survival Game was developed with the objective of demonstrating the practical implementation of data structures within an interactive real-time gaming environment. Unlike conventional survival games, this project integrates algorithmic logic—specifically the **Queue (FIFO)** data structure—into core gameplay mechanics. The system enforces that zombies must be eliminated in the order they enter the screen, simulating real-world queue processing behavior.

The game was developed using the C programming language with the **Raylib graphics library** for rendering and real-time interaction. Additionally, **SQLite** was integrated to manage user authentication and persistent high-score tracking. The project combines real-time collision detection, linked list management for bullets, database integration, and progressive level design into a complete interactive system.

User Authentication and Security

A secure login and registration system was implemented using SQLite database integration. User credentials are stored securely, and passwords are hashed before being saved to the database to enhance security. Upon successful login, users can access gameplay features and track their personal high scores.

The authentication mechanism ensures:

- Secure user registration and login validation.
- Persistent storage of high scores for each player.
- Access control to prevent unauthorized database modifications.

Queue-Based Zombie Management

The most significant feature of this project is the implementation of a **FIFO (First-In-First-Out) queue** for zombie elimination. When zombies enter the screen, they are added to a queue structure. Only the zombie at the front of the queue can be eliminated. This ensures structured processing similar to operating system scheduling and service queues.

Additionally:

- Baby zombies are managed in a separate queue after boss elimination.
- Red highlighting visually indicates the current front zombie.
- Queue advancement occurs automatically when a zombie is eliminated.

This structured enemy management system differentiates the project from traditional survival games.

Bullet Management Using Linked List

Dynamic memory allocation was used to implement bullets using a **linked list structure**. This allows:

- Efficient insertion of new bullets during gameplay.
- Safe removal of bullets upon collision or screen exit.
- Flexible memory management without fixed-size constraints.

This approach demonstrates the practical application of dynamic data structures in real-time systems.

Level Progression and Gameplay Logic

The game includes progressive difficulty across six levels:

- Level 1: Unlimited bullets.
- Level 2–5: Gradually decreasing bullet limits.
- Level 6: Survival challenge with only 2 bullets.

Boss zombies appear periodically, requiring multiple hits to eliminate. Upon defeating the Level 6 boss, a congratulatory message is displayed, marking successful game completion.

Additional gameplay elements include:

- Moving obstacles.
- Sound effects for shooting and zombie elimination.
- Pause and mute functionality.
- Restart system upon game over.

Graphical User Interface and Visual Design

The graphical interface was developed using Raylib, providing smooth rendering and real-time event handling. The system includes:

- Login screen interface.
- Game play screen.
- Game over screen with background display.
- Final congratulations screen with plain background.
- Real-time score and high-score display.

Proper screen scaling and view offset calculations ensure that the game remains visually centered even when the window is resized or maximized.

Achievements

Through the development of the Queue-Based Zombie Survival Game, several technical and conceptual milestones were achieved:

- Successfully implemented a **FIFO queue system** within a real-time game environment.
- Integrated linked list data structures for dynamic bullet management.
- Designed and implemented multi-level gameplay with progressive difficulty.
- Implemented boss and baby zombie mechanics with separate queue management.
- Developed secure user authentication using SQLite and password hashing.
- Integrated persistent high-score tracking per user.
- Implemented collision detection algorithms for player, bullets, zombies, and obstacles.
- Developed a complete GUI system with login, gameplay, and end screens.
- Enhanced understanding of real-time event handling and memory management in C.

This project significantly strengthened knowledge of data structures, especially queues and linked lists, by applying them in a practical and interactive environment. It improved debugging skills, algorithm implementation ability, and understanding of real-time graphical programming.

On a personal and academic level, the project provided hands-on experience in combining theoretical computer science concepts with practical software development. It demonstrates how structured algorithms, database systems, and graphics programming can be integrated into a cohesive and functional application.

Overall, the project represents a comprehensive application of core computer science and data structures principles within an engaging and educational gaming system.

Chapter 5

System Requirements

5.1 Software Requirements

This section presents the required software components for developing and running the Queue-Based Zombie Survival Game.

5.1.1 Minimum Software Requirements

- **Operating System:** Windows 10 or above, Linux, macOS
- **Programming Language:** C (C99 or above)
- **Graphics Library:** Raylib
- **Database:** SQLite3
- **Cryptography Library:** OpenSSL (for SHA-256 password hashing)
- **Compiler:** GCC / MinGW
- **IDE / Editor:** Visual Studio Code, Code::Blocks, or any C-compatible IDE
- **Version Control (Optional):** Git

5.2 Hardware Requirements

5.2.1 Minimum Hardware Requirements

- **Processor:** Dual-core CPU 1.5 GHz or higher
- **RAM:** 4 GB
- **Storage:** 300 MB free disk space

- **Display:** 1280x720 resolution or higher
- **Input Devices:** Keyboard and mouse
- **Audio Output:** Speakers or headphones (for sound effects)

5.3 Functional Requirements

The Queue-Based Zombie Survival Game implements the following functionalities:

- **User Authentication Module:**
  - User registration and login system
  - Password hashing using SHA-256
  - Persistent high-score storage using SQLite
- **Queue-Based Zombie Management:**
  - Zombies are inserted into a FIFO queue upon entry
  - Only the front zombie can be eliminated
  - Separate but similar queue management for baby zombies
- **Bullet Management System:**
  - Dynamic bullet creation using linked list
  - Collision detection with zombies and obstacles
  - Automatic bullet removal upon impact or screen exit
- **Level Progression System:**
  - Six progressive levels
  - Boss zombie generation after fixed eliminations
  - Bullet limitation logic for advanced levels
  - Final congratulations screen upon Level 6 completion
- **Game Control System:**
  - Player movement (W, A, S, D)
  - Shooting mechanism (Spacebar)
  - Pause functionality (P)
  - Mute/Unmute option (M)

  - Restart option (0 key)

- **Obstacle Management:**
  - Static and moving obstacles
  - Collision detection with player

5.4 Non-Functional Requirements

5.4.1 Performance Requirements

- Maintain smooth gameplay at 60 FPS
- Efficient memory management using dynamic allocation
- Optimized collision detection for real-time responsiveness

5.4.2 Security Requirements

- Secure password hashing using SHA-256
- SQLite database protection from unauthorized access
- Input validation during login and registration

5.4.3 Usability and Accessibility

- Intuitive graphical interface
- Clear display of score, level, and high score
- Centered game layout for maximized window mode
- Clear visual indicators for active (red) zombies
- Distinct game states: Login, Gameplay, Game Over, Congratulations

5.4.4 Maintainability and Scalability

- Modular design separating gameplay logic, database module, and rendering system
- Easy to extend levels beyond Level 6
- Flexible structure to add new zombie types or power-ups
- Separate queue management allowing future algorithm extensions

Chapter 6

Conclusion and Recommendation

This project successfully developed a Queue-Based Zombie Survival Game using the C programming language and the Raylib graphics library, integrated with an SQLite database for user authentication and high-score management. The primary objectives outlined in Section 1.2 have been achieved, including the implementation of FIFO-based zombie management, dynamic bullet handling using linked lists, progressive level design, and secure login functionality.

The system provides an interactive and structured gameplay environment where zombies must be eliminated following a First-In-First-Out (FIFO) queue mechanism. This ensures that the game not only serves as an entertainment platform but also demonstrates the practical application of data structures. The bullet management system dynamically allocates memory using a linked list structure, enabling efficient creation and removal of bullets during gameplay.

The integration of SQLite ensures persistent storage of user credentials and high scores, allowing players to track performance across sessions. Additionally, real-time collision detection, level progression, boss mechanics, moving obstacles, and sound integration contribute to a complete and engaging gameplay experience.

Through this project, significant hands-on experience was gained in real-time graphics rendering, memory management, event handling, data structure implementation, and database integration. The modular design of the system ensures that future enhancements can be incorporated without major structural modifications.

6.1 Limitations

While the system meets its primary objectives, several limitations exist:

- The game currently supports only single-player mode; multiplayer functionality is not implemented.
- The database is local (SQLite), and online leaderboard synchronization is not supported.

- Enemy types are limited to normal zombies, baby zombies, and boss zombies.
- The game does not include advanced animations or physics-based mechanics.
- AI movement logic is basic and does not include advanced pathfinding algorithms.

6.2 Future Enhancements

The project provides a strong foundation for further development. Potential improvements include:

- Implementing multiplayer mode (local or online).
- Developing an online leaderboard system with centralized database support.
- Introducing additional zombie types with unique behaviors and abilities.
- Implementing advanced AI pathfinding algorithms such as A* for smarter enemy movement.
- Adding power-ups, weapons, and inventory systems.
- Enhancing graphics with sprite animations and particle effects.
- Integrating background music and advanced sound design.
- Adding difficulty customization and survival mode beyond Level 6.
- Implementing cloud-based authentication and two-factor login verification.

Overall, the project demonstrates a functional, educational, and structured survival game that effectively integrates Queue data structures, linked lists, collision detection, and database management within a real-time graphical environment. It serves as a practical demonstration of theoretical computer science concepts applied in interactive software development and provides a solid foundation for advanced gaming and system design enhancements in the future.

References

Cormen, T. H., Leiserson, C. E., Rivest, R. L., & Stein, C. (2009). *Introduction to algorithms* (3rd ed.). MIT Press.

Hlavacs, H. (2017). *A procedurally generated world for a zombie survival game.* Research publication.

Motion Twin. (2008). *Die2nite.* Retrieved from https://www.motion-twin.com.

Nguyen, T. (2023). *Developing a 3d zombie survival run game with unity* (Tech. Rep.). Centria University of Applied Sciences.

OpenSSL Project. (2024). *Openssl cryptography and ssl/tls toolkit.* Retrieved from https://www.openssl.org.

SQLite Consortium. (2024). *Sqlite documentation.* Retrieved from https://www.sqlite.org/docs.html.

Stroustrup, B. (2013). *The c++ programming language* (4th ed.). Addison-Wesley.

Technologies, R. (2024). *Raylib documentation.* Retrieved from https://www.raylib.com.

Appendix

.1 Additional Figures and Screenshots

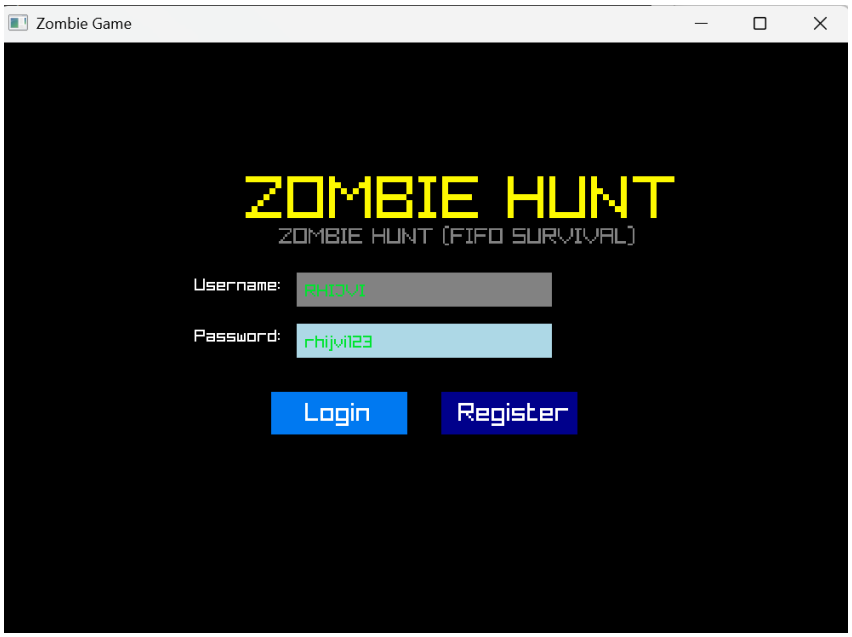

Figure A1: Login Screen of Zombie Hunt (FIFO Survival Game)

Figure A2: Gameplay - Level 1 (Unlimited Bullets)

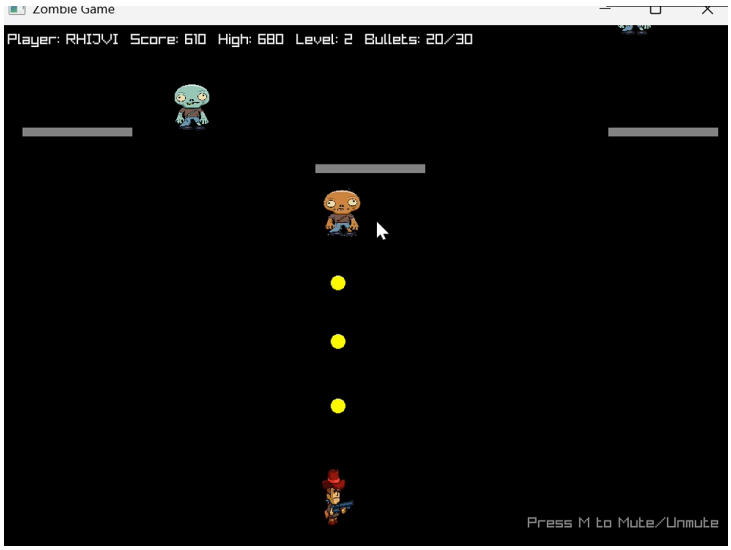

Figure A3: Gameplay - Level 2 (Limited Bullets)

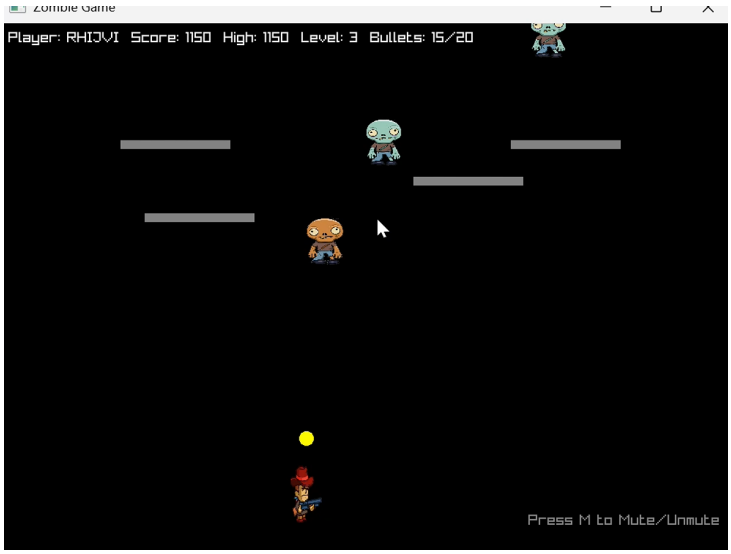

Figure A4: Gameplay - Level 3 (Increased Difficulty)

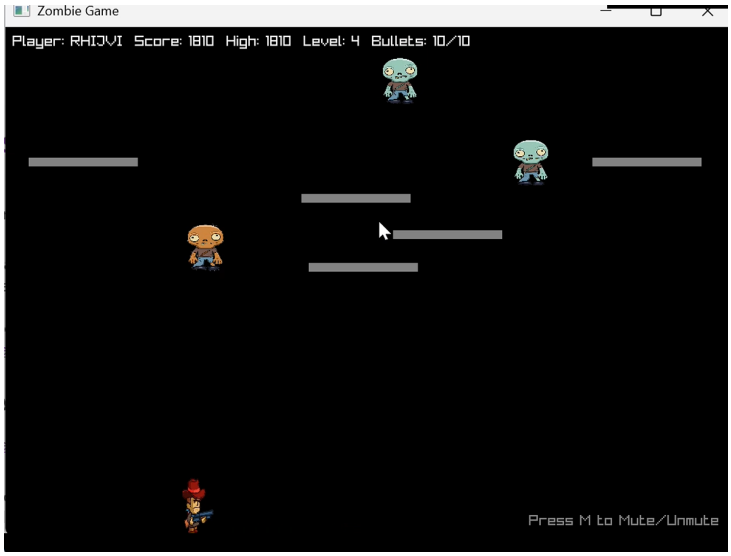

Figure A5: Gameplay - Level 4 (Moving Obstacles)

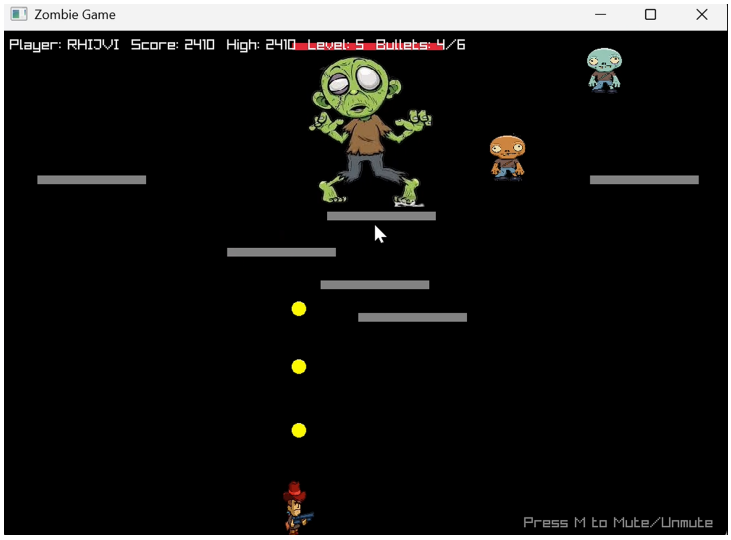

Figure A6: Gameplay - Level 5 (High Intensity Mode)

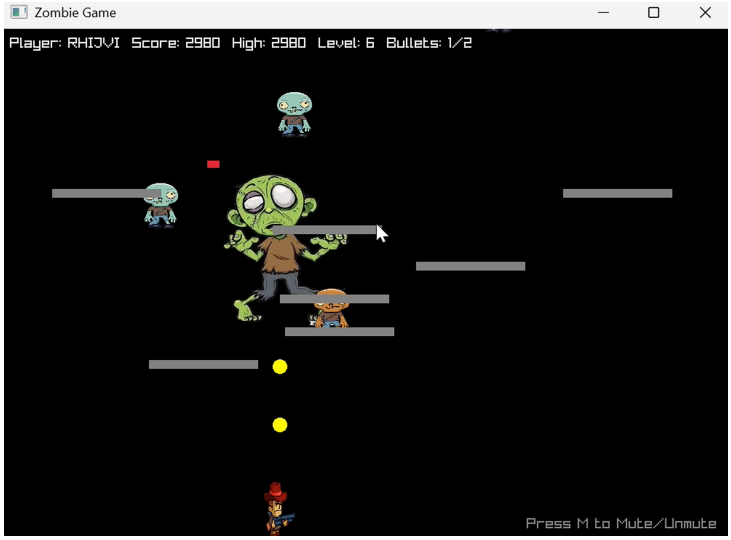

Figure A7: Gameplay - Level 6 (Final Survival Stage)

Figure A8: Baby Zombies Spawned After Boss Defeat

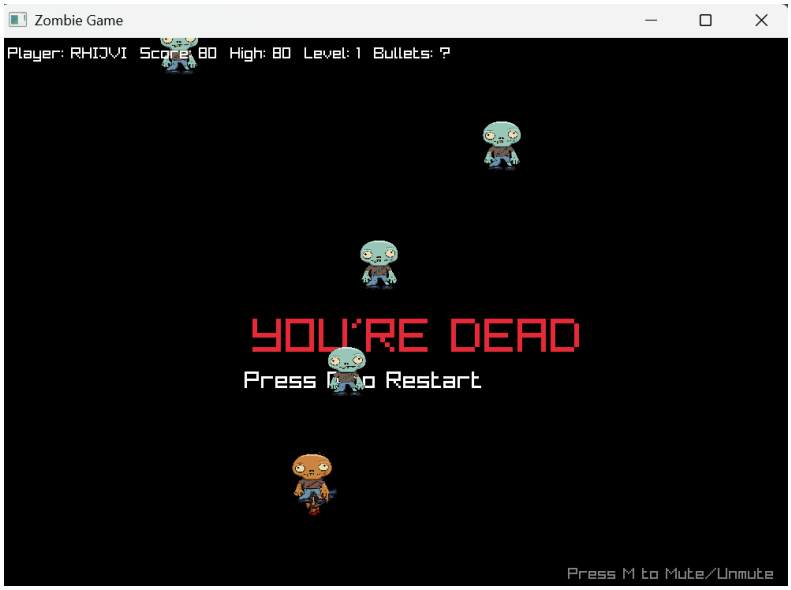

Figure A9: Game Over Screen when zombie touches player

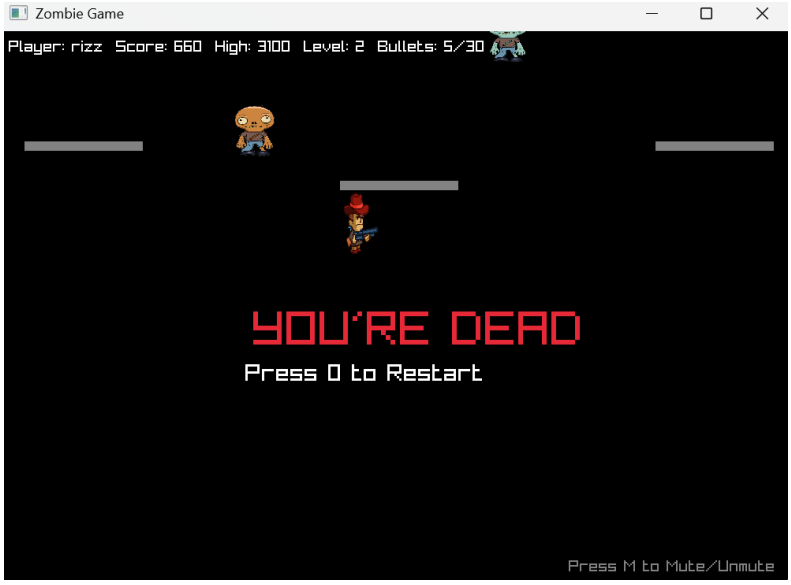

Figure A10: Game Over Screen when player touches obstacle

Figure A11: Congratulations Screen After Completing Level 6

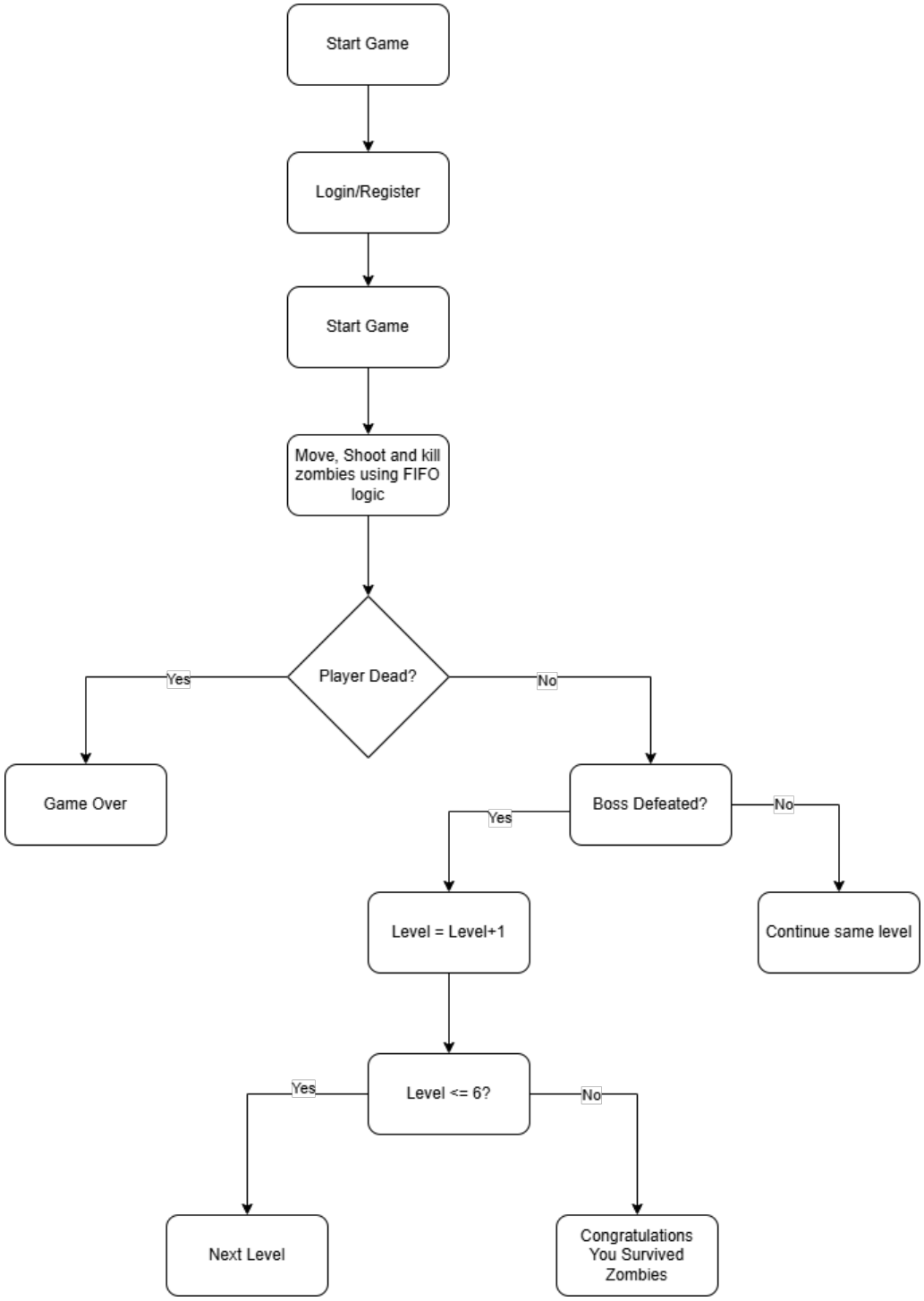

Figure A12: System Flowchart of Queue-Based Zombie Survival Game